2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA) | 978-1-6654-3333-4/21/$31.00 ©2021 IEEE | DOI: 10.1109/ISCA52012.2021.00067

NASA: Accelerating Neural Network Design with a NAS Processor

Xiaohan Ma[1,3], Chang Si[1,3], Ying Wang*[1,2], Cheng Liu[1,2], and Lei Zhang[1]

[1]Institute of Computing Technology, CAS
[2]State Key Laboratory of Computer Architecture, Institute of Computing Technology, CAS
[3]University of Chinese Academy of Sciences
{*maxiaohan, sichang19g, wangying2009, liucheng, zlei*}*@ict.ac.cn*

***Abstract*—Neural network search (NAS) projects a promising direction to automate the design process of efficient and powerful neural network architectures. Nevertheless, the NAS techniques have to dynamically generate a large number of candidate neural networks, and iteratively train and evaluate these on-line generated network architectures, thus they are extremely time-consuming even when deployed on large GPU clusters, which dramatically hinders the adoption of NAS. Though recently there are many specialized architectures proposed to accelerate the training or inference of neural networks, we observe that existing neural network accelerators are typically targeted at static neural network architectures, and they are not suitable to accelerate the evaluation of the dynamical neural network candidates evolving during the NAS process, which cannot be deployed onto current accelerators via the off-line compilation.**

To enable rapid and energy-efficient NAS in compact single-chip solutions, we propose NASA, a specialized architecture for one-shot based NAS acceleration. It is able to generate, schedule, and evaluate the candidate neural network architectures for the target machine learning workload with high speed, significantly alleviating the processing bottleneck of one-shot NAS. Motivated by the observation that there are considerable computation sharing opportunities among the different neural network candidates generated in one-shot NAS, NASA is equipped with an on-chip network fusion unit to remove the redundant computation during the network mapping stage. In addition, the NASA accelerator can partition and re-schedule the candidate neural network architectures at fine-granularity to maximize the chance of data reuse and improve the utilization of the accelerator arrays integrated to accelerate network evaluation. According to our experiments on multiple one-shot NAS tasks, NASA achieves 33.52× performance speedup and 214.33× energy consumption reduction on average when compared to a CPU-GPU system.

I. Introduction

Deep learning technology provides a way to automate the procedure of solving a complex problem by training and parameterizing a pre-built deep model with powerful computers. However, building such a predictive model still takes a lot of human effort in conventional deep learning techniques. Nowadays, neural architecture search (NAS) technology has been intensively studied to automate the design process of efficient neural network architectures for a given task [71]. NAS eliminates the need for the labor-intensive process of designing neural networks and exceeds the performance of manually designed architectures on many tasks such as image classification [6], [8]–[11], [24], [43], [53], [59], [72], object detection [17], [72] and semantic segmentation [12], [16], [39]. When the search of both model parameters and architectures becomes fully automated, deep learning will have a lower entry barrier and become accessible to users of diverse backgrounds and expertise.

The rapid progress of NAS technology enables a user-friendly and automated development flow of machine learning algorithms, but it also increases the requirement for the growth of computing power in machine learning infrastructures. Given the massive search space, typical NAS methods must iteratively sample neural network architectures from the space and evaluate the performance of generated architectures on the target task, instead of optimizing a fixed human-designated architecture. However, in this procedure, conventional NAS methods must explore the enormous number of models, and state-of-the-art NAS systems usually take thousands of GPU hours to train and evaluate the generated network architectures. For example, It consumes 2,000 GPU hours for NASNet [72] to conduct a single search process on the CIFAR-10 dataset.

The latest methodology such as one-shot NAS [6], [8], [9], [21], [24], [30], [67] mitigates the computational overhead of conventional search approaches by reducing the time spent on candidate architecture training. In one-shot NAS, all sampled architectures directly inherit parameters from a supernet and get evaluated on the target task. Essentially, one-shot NAS greatly reduces the cost of traditional NAS methods and enables the search on large datasets such as ImageNet. Nevertheless, it still needs tens to hundreds of GPU hours to find the optimal network architecture for a specific task [6]. Thus, to promote the exploration efficiency and lower the design effort of automated machine learning application, a specialized architecture that provides high-throughput and energy-efficient NAS processing solution is demanded. If the NAS acceleration solution is achievable, machine learning developers do not necessarily need cloud-scale infrastructure and distributed GPUs to build customized network architectures. Although there are many existing neural network accelerators [14], [15], [56], [61] focused on network inference or training, to design a specialized accelerator for efficient NAS remains a non-trivial task.

Firstly of all, an efficient NAS processor must handle many

*Corresponding Author.

978-1-6654-3333-4/21/$31.00 ©2021 IEEE
DOI 10.1109/ISCA52012.2021.00067

heterogeneous network architectures in parallel to achieve high-throughput search. Fortunately, the mechanism of one-shot NAS brings new opportunities to the accelerator design. Although the candidate neural network architectures are heterogeneous, there exist some common sub-networks among these candidates due to the structure sharing feature of one-shot models. Depending on where they appear, the identical sub-network structures inside different architecture candidates provide two acceleration opportunities worth exploiting: cross-model computation sharing and cross-model data reuse. However, the opportunities require the ability of the accelerator to partition, schedule, and run the dynamically generated networks simultaneously and to reuse the intermediate results and parameters between these network candidates.

Second, compared to conventional neural network processors, a NAS processor must be able to generate and evaluate the performance of many dynamic and heterogeneous neural network architectures that keep evolving rapidly. Hence, how to support dynamically changing neural network architectures is a key problem in the architecture design of NAS processor. A typical neural network accelerator needs a compiler to pre-deterministically map the static graph of neural network architecture to the underlying computation resources, and sophisticated methods are implemented to find an optimized instruction flow and mapping scheme [13]. In such processors, the time cost of compilation processing and setup are amortized over numerous input instances. With regard to NAS, the rapidly changing architectures with a fixed size of validation dataset cannot afford the cost of software compilation and setup, and requires an on-line components to deal with instruction generation and the scheduling-level optimization for many dynamic networks.

This work presents a specialized processor for one-shot based network architecture search, NASA, a high-throughput accelerator including an evolutionary search unit and a multi-tile accelerator to evaluate dynamic generated neural network architectures quickly. By running multiple candidate neural network architectures simultaneously, NASA can exploit the shared sub-networks of the dynamic network candidates, eliminate the resulted computational redundancy between them, and takes full advantage of cross-model computation sharing and cross-network data reuse.

Specifically, this paper makes the following contributions:

1) We design and implement NASA: a specialized high-throughput accelerator for automatic network architecture search. The accelerator architecture provides a solution to fast one-shot NAS on a single chip. To our knowledge, NASA is the first accelerator targeting at NAS and automated neural network design.

2) We identify the opportunities for cross-model computation sharing and cross-model data reuse unique to one-shot NAS algorithms, which provide insights on designing efficient NAS architecture.

3) We introduce an optimized workflow for one-shot NAS, which includes a network fusion algorithm and an operation scheduling algorithm to exploit cross-model computation sharing and cross-model data reuse respectively.

4) We implement a hardware-based mapping unit to exploit two types of cross-model optimization and quickly convert the candidate NN architectures to accelerator instruction.

5) We implement and evaluate NASA on state-of-the-art one-shot NAS methods and different target datasets. Compared to a CPU-GPU system running the same methods and datasets, NASA achieves a 33.52× improvement in performance and 214.33× improvement in energy efficiency on average.

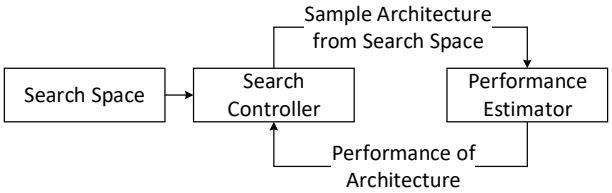


Fig. 1: Components in a typical NAS method.

II. Background

A. Neural Architecture Search

Figure 1 illustrates the process of NAS. The search controller defines the search strategy that details how to iteratively generate new architectures from the search space. The performance estimator then evaluates the performance of the generated architecture and returns its performance to the search controller. To converge on optimized architecture, the search controller tunes its search strategy on the basis of the received performance feedback.

Network Search Space. Typical search spaces include the chain-structured network [3], multi-branch network [8], [27], [58], [59], and cell-based network [10], [16], [40], [51], [53], [72]. A chain-structured network is a sequence of neural operations. A multi-branch network introduces skip-connections and enlarges the search space. A cell-based network is constructed with stacking cells where each cell is a multi-branch network but only consist of 4-10 neural operations.

Architecture Search Strategy For efficient search over the space, the search strategy implemented in the search controller plays a key role in NAS. Various search strategies including random search [6], [36], evolutionary algorithm [27], [43], [53], [58], reinforcement learning [3], [5], [51], [71] and gradient-based methods [25], [40] have been investigated for different search spaces.

Architecture Estimation The performance estimator evaluates the candidate neural network architectures. Originally, it needs to train each network and then measures the validation accuracy. Since training a network from scratch is computationally expensive, the evaluation of a network architecture candidate is time-consuming. Worse still, the entire NAS procedure usually needs an evaluation of thousands of different architectures before it converges on an optimized neural architecture. Thereby, NAS is extremely computationally expensive, and the search overhead is prohibitively high.

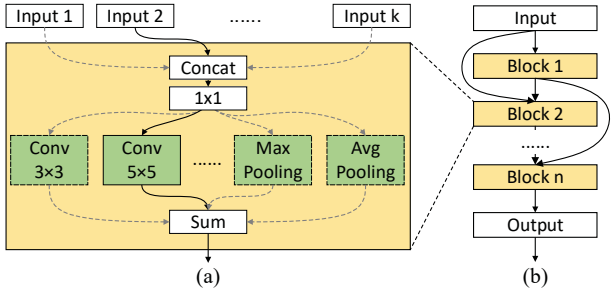


Fig. 2: An example of one-shot supernet.

B. One-Shot NAS

Since training is required for each candidate under evaluation in the original NAS, it becomes the bottleneck and hinders the adoption of NAS. To address the problem and enable a practical NAS solution, one-shot methods [6] are proposed to train a supernet comprised of all the optional network operations only once in a NAS task. Then all the candidates are generated by sampling an optional path from the supernet. The candidates do not need separate training in evaluation and directly inherit parameters from the supernet. Although the parameters inherited from the supernet are not optimal for each candidate, it provides a proxy to rank architectures in the search space. Compared to the baseline NAS, the search time of one-shot NAS is reduced significantly due to the sharing of a single supernet, and one-shot methods are considered to be one of the most cost-effective NAS approaches [30].

As shown in Figure 2, the supernet of the one-shot method is composed of a series of choice blocks (CB) that include multiple basic neural operations such as 3×3 convolution, max pooling, or complex sub-networks. Only one operation in each CB will be selected during the search process, while the remaining operations will be bypassed. The inputs to a given CB are also selected from the outputs from previous CBs. The number of choice operations in each CB, N_{choice}, the number of inputs of each CB, N_{input}, and the number of CBs, N_{block}, are the three major hyper-parameters of one-shot methods. Chain-structured network and cell-based network are two fundamental supernet types used in one-shot NAS. Chain-structured supernet supports direct model learning on large dataset and enables the search for model hyper-parameters such as the number of filter channels and mixed-precision quantization, thus gained more popularity than the cell-based supernets [7], [9], [11], [30], [42], [63]. Since one-shot NAS usually need to train only one single supernet and reuses it to search architectures for different use cases, the cost of supernet training will be amortized while the search efforts cannot be avoided [9]. Recent work [9] has also shown that the supernet's weight can be directly used as the final search result, so that the time-consuming process of model retraining and fine-tuning will be eliminated, which makes weight-sharing supernet a promising approach towards automated model design.

The typical search strategy of one-shot methods includes the random search [6] and the evolutionary algorithm [9], [21], [30]. Due to the large search space of the one-shot method, evolutionary algorithm is more efficient and converge faster. A typical one-shot search space has 5-40 CBs. Hence the evolutionary algorithm also use encoded genes of equal size to represent the CBs, and it generates NN candidates through the classic genetic algorithm.

Despite the improvement over early NAS frameworks, the computing overhead of one-shot methods remains a significant challenge to the computing systems, especially for the applications with large models and datasets. For instance, OSAS [6] took around 80 GPU hours to evaluate all 20,000 architectures in a single search.

III. Motivation

In this section, we mainly illustrate the challenges of accelerating one-shot NAS with existing DNN processors and explore the unique acceleration opportunities within one-shot NAS.

A. Challenges to One-shot NAS on DNN Accelerators

Deploying neural network (NN) models onto a DNN accelerator usually requires an off-line compiler. The compilation will parse the NN model and explore many network-scheduling options to make the best use of the accelerator resources, and thus it is usually time-consuming. For example, Jetson AGX Xavier [47] with two NVDLA acceleration cores [46] can perform MobileNet [32] inference on 10,000 images within 8.35s, but the compilation of the MobileNet model for that processor takes 11.325s on a 12-core X86 CPU running at 2.9 GHz. As the NN models are usually static and the compilation results, i.e., the instructions for a specific DNN accelerator, can be reused, thus, the off-line compilation time can be amortized by the numerous potential inference. However, in one-shot NAS, the NN candidates to be evaluated are strategically generated and evolve rapidly during the search process. If each candidate invokes an independent compilation stage before being evaluated on the target validation dataset, the total compilation time taken by all the candidates will dominate the entire search process in NAS.

To avoid the frequent compilation of different NN architectures, the NAS accelerator that allows instant and efficient deployment of dynamically changing NN architectures is highly demanded. Furthermore, to ensure performance-efficiency NAS, this accelerator must be able to conduct on-line scheduling and block-level optimization for the numerous heterogeneous NN architectures at high-speed and high-efficiency, even when the off-line network compiler is absent.

B. Opportunity of Computation Sharing

Although the generated NN candidates in one-shot NAS are different, they are closely related. First of all, these NN candidates share the same supernet as stated in Section II-B. The number of choices, i.e. N_{choice} in each CB is small, usually ranging from 4 to 10, so the types of CBs are limited. In contrast, the number of NN candidates in a one-shot search is much larger and can be up to thousands, which indicates that

many CBs are identical. The CBs with the same configuration essentially have the same weights, contributing to data reuse. In addition, all the NN candidates are evaluated on the same fixed validation dataset and share the same input data. Thereby, many computation results of the CBs can be reused given both the same inputs and CB configurations.

An example of the computation and data reuse is presented in Figure 3. Figure 3(a) shows three typical NN candidates generated in one-shot methods. Note that they are executed from bottom to top. Suppose the search space includes 6 CBs and N_{choice} is 4. The digits in each CB refers to a specific configuration. In the beginning, all the three candidates invoke choice 1 of CB-1. Since the input of CB-1 is the same for different candidates in Figure 3(a), the three networks perform identical operations in CB-1 and produce the same output consequently. If the networks are scheduled together, CB-1 needs to be executed only once and the result can be reused among the three networks, which is deemed as Cross-Model Computation Sharing in this work. Similarly, the results of CB-2 can also be reused. Afterward, network-1 chooses CB-3 of a different configuration while network-2 and network-3 continue to share the computation result. For CB-4, network-1 still has to execute the CB separately even though the CB configuration is the same as that of network-2, because they have different input due to prior computation path divergence. Accordingly, there will be no computation sharing for the following CBs because they all have different input.

To eliminate the redundant computation, we may fuse the three NN candidates as shown in Figure 3(b). In this small example, there are 18 CBs in the original setup. If the redundant computation can be fully removed, we only need to process 13 CBs and the amount of the computational load can be reduced by 27.78%. We also reproduce and analyze the probability of computation redundancy in a practical one-shot NAS example [30]. In this example, 50 NN candidates are generated in each iteration by an evolutionary search controller. The analysis result is presented in Figure 4. We observe that the proportion of redundant operations gradually increase as the search proceeds, and 29.32% of operations are redundant on average if all the 50 networks are fused. This observation reveals that there is considerable cross-model computation sharing to be exploited in one-shot NAS.

C. Opportunity of Data Reuse

On top of computation sharing, we also observe that many other CBs from different NN candidates may share the same operands, i.e., weights which they inherit from the same supernet. As shown in 3(c), the CBs in gray located in different NN candidates share the same weight. To be more specific, the weights of CB-4 and CB-5 in gray are shared by network-1 and network-2, while the weights of CB-6 in gray is shared by network-2 and network-3. Although they cannot share the computation due to different input features, these CBs can be scheduled for execution in parallel such that the shared weights are loaded only once from the external memory and reused. The parallel execution of CBs with the shared data also can

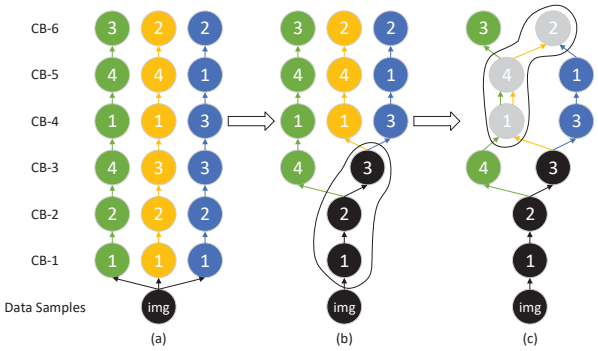


Fig. 3: An example of network fusion. (a) original networks; (b) Cross-Model Computation Sharing; (c) Cross-Model Data Reuse.

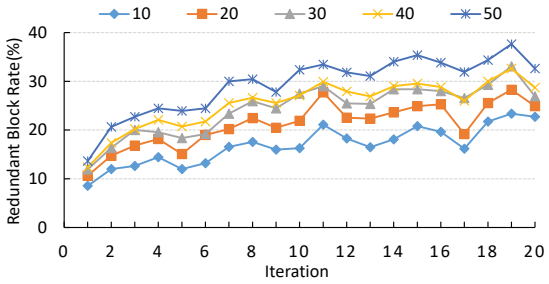


Fig. 4: Percent of redundant blocks in different iterations of one-shot search and different number of fused networks.

be considered as Cross-Model Data Reuse. Compared to the original NAS implementation that executes the NN candidates independently, Cross-model data reuse can potentially reduce the memory accesses significantly when the cross-model data reuse is explored. Conclusively, the discovered phenomenon of computation sharing and data reuse poses a great opportunity for the NAS processor design.

In summary, the NN candidates that vary frequently during one-shot NAS not only introduces challenges but also opportunities for us to create more efficient processor architecture for efficient NAS acceleration.

IV. NASA Workflow

In this section, we introduce an optimized workflow for one-shot NAS. As shown in Figure 5, the workflow is an evolutionary algorithm based iterative process in which the initial population of NN candidates is gradually improved. In order to enable high-throughput evaluation of the NN candidates, we have an additional network mapping phase that exploits the cross-model computation sharing and data reuse among the heterogeneous NN candidates through network fusion and operation scheduling.

A. Network Generation

The goal of the network generation phase is to discover diverse NN candidates with more competitive performance. To achieve this goal, we employ the genome encoding method to

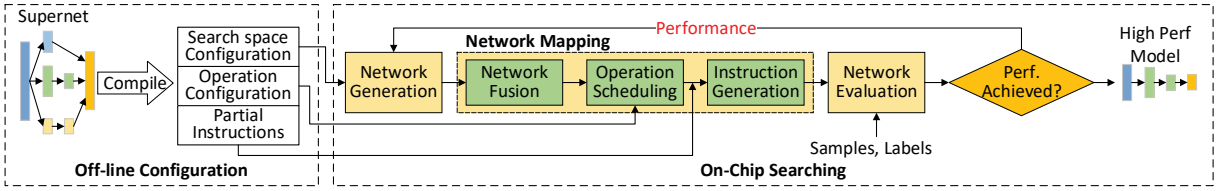


Fig. 5: Workflow of NASA.

represent heterogeneous NN candidates and the corresponding evolutionary operations, crossover and mutation, to explore the search space efficiently.

Genome Encoding. NASA uses two types of genes to describe a CB: a block gene to describe the CB configuration and a connection gene to describe the input connectivity of the CB. The block gene has two attributes {`choice`, `number`} where the `choice` describes which operation is selected by this CB and the `number` describes the number of input connections to this CB. The connection gene has one attribute {`src_id`} which describes the source CB ID of this connection. Each gene has two metadata {`net_id`, `cb_id`} which describe the genome and the CB to which the gene belongs.

Network Evolution. The two evolutionary operations used in network generation are crossover and mutation. Crossover aims at inheriting the sub-structures of superior parents and exploring new architectures by recombining sub-structures from different parents. In the crossover operation, the gene of a new NN candidate is created by randomly picking attributes from two parent genes. Mutation aims at increasing the diversity of network architectures. Akin to biological mutation, the mutation operation randomly changes the attributes of genes.

B. Network Mapping

In the iterative search process, the network mapping phase essentially maps the population of NN candidates generated in the network generation phase onto the DNN accelerator for efficient performance evaluation. To begin with, it has a network fusion step to remove the redundant CBs among different NN candidates, in order to exploit cross-model computation sharing as observed in Section III-B. Then an operation scheduling step is taken to resolve the dependencies between operations from the fused network, identifies the operations with shared data and batches them together as much as possible to enhance on-chip locality and reduce outstanding memory requests. At last, an instruction generation step fetches the scheduled operations, dynamically generates the corresponding accelerator instructions for them, and issues the instructions to the network evaluation stage.

1) Network Fusion: In this step, we present how to exploit cross-model computation sharing to eliminate repetitive operations between the NN candidates.

The population of generated NN candidates can be represented as a set of directed acyclic graphs (DAGs) $\mathcal{G} = \{G_1(V_1, E_1),\ G_2(V_2, E_2),\ ..., G_n(V_n, E_n)\}$, whose vertices are the CBs and edges are the connections between the CBs. Discovering cross-model computation sharing is to search for

Algorithm 1: Network Fusion

Input: N networks $\{G_1(V_1, E_1), G_2(V_2, E_2), ..., G_N(V_N, E_N)\}$, each contains M vertices
Output: A fused network FG

```
for vid ← 0 to M do
    for nid ← 0 to N do
        v ← V_nid[vid]
        foreach incoming edge(u, v) of v do
            if u is not a shared vertex then
                add v and all incoming edge(u, v) of v to FG
                continue
        for u ← VQ do
            if attr(v) = attr(u) and indegree(v) = indegree(u) then
                share_flag ← True
                foreach incoming edge(v', v) of v do
                    if not exist edge(v', u) then
                        share_flag ← False
                        continue
                if share_flag = True then
                    for all edge(v, *), update v to u
                    mark u as a shared vertex
                    continue
        if share_flag = False then
            add v to VQ
    foreach v ∈ VQ do
        add v and all incoming edge(u, v) of v to FG
```

shared sub-graphs from these DAGs and fuse them into one fused network as shown in Figure 3(b). The vertices v_1 and v_2 from G_1 and G_2 are shared if and only if:

1) $attribution(v_1) = attribution(v_2)$,
2) $indegree(v_1) = indegree(v_2)$,
3) for each incoming $edge(u, v_1)$ of v_1 in G_1, there exists $edge(u', v_2)$ in G_2, and u, u' are shared,

where *attribution()* returns the attribute of the CB, *indegree()* returns the number of its incoming edges, and *edge(u, v)* is the edge from *u* to *v*.

Thanks to the ordered vertices and the directed edges in

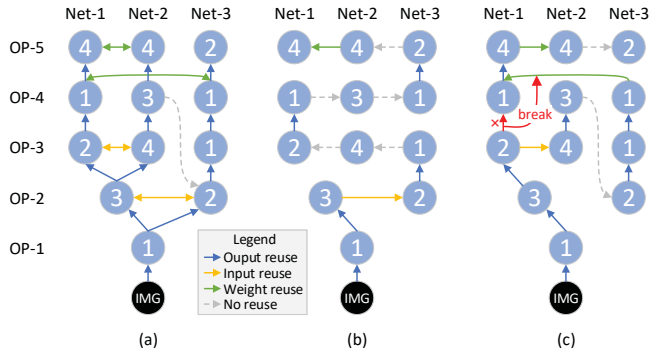


Fig. 6: An example of scheduling. (a) scheduling graph, (b) BFS scheduling, (c) greedy scheduling.

DAGs, we can compare the attributes and the incoming edges of the vertices one by one in the DAG pairs to find the shared sub-graphs. Therefore, the time complexity of comparing two DAGs is $O(V + E)$, wherein V and E are the number of vertices and edges, respectively. However, when it comes to multiple DAGs, finding all the shared sub-graphs requires comparing every possible pair from the DAGs. Thus the time complexity becomes $O(M(V + E)^2)$ where the M is the number of DAGs. Fortunately, we observed that vertex v will not be shared by networks if any of its source vertices is not shared. Therefore, we propose a bypass strategy to quickly filter out most of the non-shared vertices, i.e. unique vertices. The bypass mechanism checks whether the sources of a vertex are shared, and it will add the pre-determined non-shared vertex into the fused network directly without comparison.

With the bypass strategy, the network fusion procedure is shown in Algorithm 1. In the procedure, the algorithm will processes all the vertices with the same id in the networks. At first, the proposed bypass strategy filters out most of the non-shared vertices (*line 4-7*). For a potential shared-vertex, the algorithm compares it with the vertices one by one from other networks (*line 8-18*). If a vertex u is found to have the same attributes and edges as vertex v, the vertex v is deemed redundant and will be fused into vertex u rather than being added to the fused network. After that, the corresponding edges will be updated (*line 16*) and the u is logged as a shared vertex (*line 17*). Otherwise, vertex v will be added to the comparison queue VQ and wait for comparison to the subsequent vertices (*line 19-20*). After all vertices with the same id are processed, the vertices in VQ are deemed unique, and they will all be added to the fused network (*line 21-22*). Then the algorithm proceeds to the vertices of the next id.

2) Operation Scheduling: After the vertices of CBs in the network candidates are fused into a unified network, the CBs are decomposed into basic operations, e.g. convolutions, full-connection and etc., which are also the fundamental unit in scheduling. The operation scheduling step resolves the dependencies between operations at first, and then leverage a greedy based policy to re-order the dependency-free operations, to fully exploit on-chip data reuse.

Algorithm 2: Operation Scheduling

Input: A scheduling graph G
Output: A scheduling path P

```
1  free_vertex ← ∅
2  group ← ∅
3  v ← root_vertex
4  next_v ← root_vertex
5  while |P| < |G| do
6      if edge(v, next_v) is a h_edge then
7          group.append(v)
8      else
9          P.append(group)
10         group ← ∅
11     foreach outgoing edge(v, u) of v do
12         if edge(v, u) is a v_edge then
13             u.dependency ← u.dependency − 1
14             if u.dependency = 0 then
15                 free_vertex.append(u)
16     max_weight = 0
17     for u ∈ free_vertex do
18         if weight(v, u) ≥ max_weight then
19             break_flag ← False
20             foreach outgoing edge(u, u') of u do
21                 if edge(u, u') is a h_edge and
                     weight(u, u') > weight(v, u) and
                     u' ∉ free_vertex then
22                     break_flag ← True
23                     continue
24             if break_flag = False then
25                 max_weight = weight(v, u)
26                 next_v = u
27     v ← next_v
```

The greedy scheduling policy not only attempt to maximize the data reusability among the dependency-free operations, but also to fully utilize the available on-chip computing resources. In this work, we define three different types of available data reusability in a fused network:

1) output reuse: when an operation is completed, its output stays on-chip as the input to the next operation.
2) input reuse: two operations share the same data as input.
3) weight reuse: two operations share the same weight.

Therefore, when an operation is scheduled, how to select the subsequent operations for computing will result in the variation of data reuse.

With these three types of data reusability, we can build a scheduling graph as shown in Figure 6(a). The vertices of the graph represent operations, while a directed edge between two vertices represents the execution order. The weight of the edge refers to the size of data that can be reused between operations,

which is also equivalent to the number of memory accesses that can be reduced, when two operations are scheduled in sequence. For clarity, we do not draw the edges with a weight of 0 in the graph. Obviously, different scheduling paths will cause different data reusability. Intuitively, simply adopting the depth-first search (DFS) or breadth-first search (BFS) cannot produce the reuse-optimal schedule. For example, Figure 6(b) shows the scheduled execution path based on BFS, which cannot fully exploit different types of data reuse. On the contrary, Figure 6(c) shows an optimized scheduling path in which all three types of data reuse are utilized.

For the optimized path, suppose we add a terminal vertex to the scheduling graph and link the last operation of all networks to it, the scheduling path is actually a Maximum Hamiltonian Path, which makes a well-known NP-hard problem. In order to find a near-optimal scheduling path on-line, NASA uses a greedy based heuristic algorithm to find the scheduling path quickly and efficiently as shown in Algorithm 2. We divide edges into two categories in a scheduling graph: vertical edges (v_edge) and horizontal edges (h_edge). The horizontal edges only represent data reuse, while the vertical edges also indicate the dependencies between vertices. A vertex is dependency-free when all source vertices of its incoming v_edges are already in the scheduling path. Since only dependency-free operations can be added to the scheduling path, the algorithm maintains a dependency-free operation queue. Every time an operation is added to the scheduling path, the algorithm uses it to update the dependency-free operation queue (*line 10-14*). Then the algorithm selects the operations that contribute to the greatest degree of data reuse from the dependency-free operation queue, and attaches it to the scheduling path (*line 16-25*). However, such a simple greedy strategy may ignore certain weight reuse opportunities. For example, in Figure 6(c), if we schedule the OP-4 of Net-1 immediately after OP-3 of Net-1 has been scheduled, OP-4 in Net-1 can no longer share weights with OP-4 of Net-3 because OP-4 of Net-3 is not dependency-free and cannot be scheduled after OP-4 of Net-1. Hence, it is necessary to check in advance and skip the operation that ruins the potential weight reuse of other operations in scheduling (*line 19-25*).

3) Instruction Generation: After the operation scheduling decision has been made, NASA generates instructions for the sorted operations. Unlike the traditional compiling flow for the neural networks with a static computation graph, NASA leverages a dynamic instruction set architecture (DISA) to achieve dynamic instruction generation. Formally, we define a DISA instruction as a 3-tuple $(Op, Config, Operand)$ where Op is the operation type, and $Config$ is a set of static parameters such as stride and kernel size, and $Operand$ is a set of dynamic tensor parameters which include the dimensions and buffer address of tensors used by the operation. At the off-line compilation stage, NASA pre-generates partial instructions for each neural operation of CBs defined by the corresponding NAS search space. In the generated partial instructions, the Op and $Config$ fields are determined, while the $Operand$ of instructions can only be assigned on-line after operation scheduling.

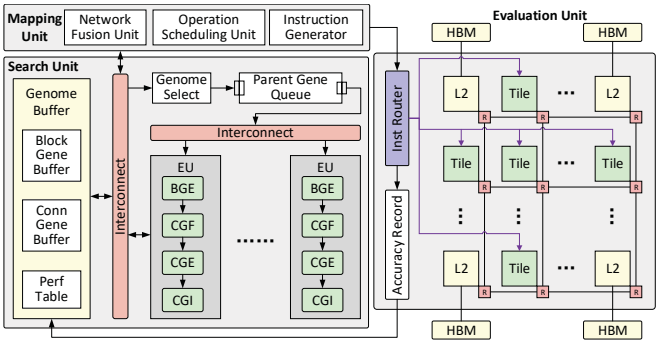


Fig. 7: Overview of NASA architecture: Search Unit, Mapping Unit and Evaluation Unit.

C. Network Evaluation

The network evaluation stage is responsible for estimating the performance of each generated NN candidate. NASA firstly performs the NN inference on the target validation datasets according to the instructions generated during the network mapping phase. Then the predictions are collected and compared to the ground truth, so that NASA can record the number of positive predictions and further obtains the result of network accuracy after all the validation datasets are processed. The overall prediction accuracy for each NN candidates will be sent to the network generation phase, and they will be utilized to direct the search for new NN candidates.

V. NASA Implementation

A. Overview

NASA is a hardware architecture that essentially implements the one-shot search workflow stated in the section IV. As shown in Figure 7, the accelerator consists of a search unit, an evaluation unit, and a mapping unit. The search unit is responsible for discovering diverse NN architectures towards better performance through an evolutionary algorithm. It uses a set of evolution units (EU), which generate new genes by crossover and mutation operations. A genome selection unit fetches the parent genomes that have high performance and then extract genes from them. Each parent gene is firstly inserted into the parent gene queue (PGQ), from which each EU dequeues parent genes to produce new genes. Once a new gene is generated, the EU sends the gene to form genomes in the genome buffer, which stores all genomes used in a search iteration. The evaluation unit is responsible for evaluating the performance of NN candidates. It is a tiled accelerator with multiple NN engines that communicate with each other through a mesh network-on-chip (NoC). It supports a coarse-grained parallel execution of multiple neural operations.

As stated in Section III-A, we identify that compiling NN architectures can be one of the bottlenecks in one-shot search. Hence, NASA is equipped with an on-chip network mapping unit to quickly parse network structures from genomes and

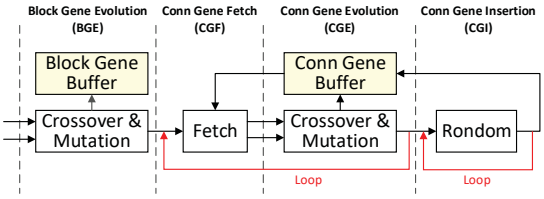


Fig. 8: Evolution Unit (EU) in Search Unit.

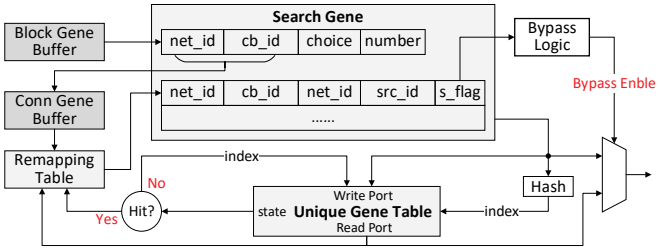


Fig. 9: Architecture of Network Fusion Unit.

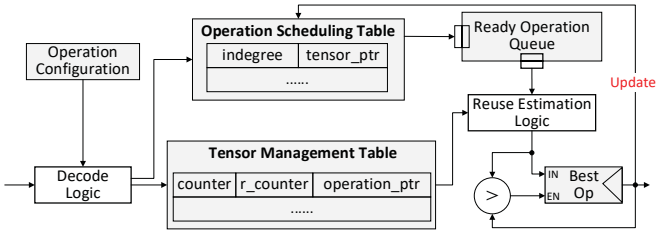


Fig. 10: Architecture of Operation Scheduling Unit.

generate instructions for them. The mapping unit also exploits the potential computation reuse and data reuse opportunities among the heterogeneous NN candidates at run-time. For computation reuse, a network fusion unit checks each CB and fuses the CBs with the same operation. For data reuse, an operation scheduling unit dynamically selects and issues the operations that contribute to the maximum data reuse.

B. Search Unit

Genome Buffer. The genome buffer stores all the genomes used in a search iteration. Specifically, the buffer is partitioned to hold the following data: (1) a block gene buffer that holds all the block genes, (2) a connection gene buffer that holds all the connection genes, and (3) a performance table that holds the performance of each genome.

Genome Selection. The genome selection unit uses a deterministic tournament selection strategy [45] to refine high performance individuals as parents for the next generation. First, a random subset of genomes is picked from the population without replacement. Then the genome with the highest performance within the subset is selected as a parent. When two parents are selected, their genomes are split, and block genes with the same `cb_id` are inserted into the PGQ at the same time.

Evolution Unit. Figure 8 shows the 4-stage pipeline design of the evolution unit. When two parent block genes are dequeued from the PGQ, EU first applies crossover and mutation to them to have a new block gene and save it back to the block gene buffer. According to the `number` of the newly generated block gene, EU fetches connection genes from the connection gene buffer and apply crossover and mutation to them. EU continues to generate new connection genes until the `number` of newly generated block genes is reached, or there is no parent gene available. If it is the second case, EU will randomly generate the remaining connection genes. Note that crossover receives two attributes and pick one from them, while mutation randomly changes the value of target attributes.

C. Mapping Unit

Network Fusion Unit. Figure 9 shows the microarchitecture of the network fusion unit (NFU) which essentially implements the Algorithm 1. NFU processes CBs with the same `cb_id` each time and stores genes of all non-shared CBs into unique gene table (UGT). For each CB, NFU firstly fetches the block gene and its connection genes to the search gene register. Since directly changing the gene code in the genome buffer will cause mistakes in subsequent network evolution iterations, the `src_id` of connection genes fetched from the genome buffer will re-encoded by referring to a remapping table which is indexed via `net_id` and `cb_id`. In addition, the re-encoding process also adds a `s_flag` which indicates whether the source CB is shared. Then the bypass logic checks the `s_flag` of all connection genes. If any gene has a false `s_flag`, the according CB cannot be shared, and all its genes can be directly output to the next stage. Otherwise, this CB needs to be compared with other CBs in the UGT. For fast comparison, we implement UGT as a hash table. If a hash table miss occurs, the current CB is also considered non-redundant, and it is inserted into the UGT. Otherwise, the current CB is redundant, and the hit entry of CB shares the same operations with the current CB. In this case, we will not add the current CB to the UGT, but allocate a new entry in the remapping table to direct the computation reuse.

Operation Scheduling Unit. Figure 10 shows the microarchitecture of the operation scheduling unit (OSU). To enable fine-grained operation scheduling without breaking the inter-dependency, OSU includes an operation scheduling table (OST), a tensor management table (TMT), and a best-op register. Each entry of OST has a `indegree` which indicates the number of pending input tensors of this operation. Whenever an operation is scheduled, OSU decreases the `indegree` of all successive operations by 1, and inserts the operation to the ready operation queue if `indegree` become 0. TMT tracks the information of tensors used in NN inference. Since there are many operations sharing the same tensors, each entry of TMT has a `counter` to track how many operations use this tensor and a `r_counter` to track how many of them are ready to be scheduled. The best-op register is used to record the operation that share the most data reuse with the previously scheduled operation, as well as the reuse size. Before scheduling a new operation, the register is initialized to zero, which represents the minimum data reuse amount. When scheduling

begins, the reuse estimation logic traverses the ready operation queue and estimates the volume of reusable tensors between the ready operations and the previously scheduled operation. If the data reuse metric is higher than the current value in register, OSU updates the best-op register. When estimating the size of reusable tensors by a candidate operation, we also check its input tensors and weight tensor in case of ruining potential data reuse of other operations. If the size of any of these tensors surpasses the current data reuse value in the best-op register and also `r_counter < counter`, there exists a potentially better data reuse chance, and OSU will skip this operation. After all the ready operations are checked, OSU sends the operation in the best-op register to a scheduled operation queue.

Instruction Generator. Operations in the scheduled operation queue are ready to be dispatched by the instruction generator (IG). At first, IG will aggressively dequeue and issue multiple independent operations within the constraint of issue width, which is decided by computing resources in the evaluation unit. Then IG allocates the computing tiles from the evaluation unit to the issued operations based on the computation overhead of each operation. Within each operation, an operation decomposition unit divides the operation into sub-operation to be processed on accelerator tiles. The operation decomposition unit fetches parameters of operand tensors from the TMT and splits them by output channels into sub-operation. Then the pre-compiled partial instruction for that operation will be fetched and its empty operand fields will be assigned dimension parameters and memory addresses of the split tensors, which is a necessary step in dynamic instruction generation.

D. Evaluation Unit

The evaluation unit is responsible for batch-processing the NN candidates on the target dataset and to measure their performance based on the execution results. Since the execution of a large number of NN candidates is time-consuming, we have a tiled accelerator to enable parallel NN execution.

NN Tile. Each tile is an Eyeriss-like [15] NN engine which consists of an array of processing elements (PEs) connected in a 2D mesh topology and an on-chip buffer to store weight and feature map. Each PE contains a small register file and a multi-function ALU that can perform multiply-accumulate (MAC), multiply, add, and compare which are required for general neural network processing.

Network-on-Chip (NoC) architecture. We have two independent NoCs implemented in the evaluation unit for on-chip data communication and instruction distribution, respectively. For the on-chip data communication pattern such as the parameter sharing and intermediate result reuse, a 2D-Mesh NoC is used to handle high-bandwidth point to point transmission. For instruction distribution, we utilize a star network to route the generated instructions from the mapping unit to any specific tile at a fixed latency.

Accuracy Recorder. After network inference completes, its result will be verified in the accuracy recorder. To exemplify with the image classification task, in the accuracy recorder, the labels of the validation images are pre-loaded and used as references to determine if the prediction of an NN candidate is true for each validation sample. Accordingly, the corresponding prediction counters of each NN candidate in the accuracy recorder can be updated. When the entire validation dataset has been evaluated, these counters are used to calculate the prediction accuracy of the NN candidates, which will be sent to the genome buffer for NN candidate generation in the next search iteration.

VI. Experimental Results

A. Experimental Setup

Benchmark. We have NASA evaluated on three one-shot models, ENNAS [69], SPOS [30], and FairNAS [21]. Table I summarizes the configurations of the target one-shot NAS frameworks mapped onto the NASA accelerator. For each one-shot model, we employ it to design networks for three different-scale image dataset, CIFAR-10 (C) [35], Tiny-ImageNet (T) [60] and ImageNet (I) [23]. We extract 10,000 images from each dataset as the testset to evaluate the accuracy of each network generated in the search iterations. For all these frameworks, we use the evolutionary algorithm to perform 30 iterations of search, and the configuration of the evolutionary algorithms is consistent with the setup of original benchmarks reported in prior paper.

Baseline Systems. We use a CPU-GPU system as the baseline. The CPUs are two 12-core Xeon E5-2650 v4 processors running at 2.20 GHz, and the GPU is NVIDIA Titan V, which provides a peak performance of 13.8 TFLOPS. NVIDIA Titan V has a total of 12 MiB on-chip memory and 12 GiB HBM2, which provide 653 GB/s memory bandwidth. For the baseline, We implement the benchmarks with PyTorch 1.6.0 [50], using the recent version of CUDA 10.1 [49] and cuDNN 7 [48]. In the baseline, GPU is responsible for DNN inference and evaluation, while the CPU runs the search algorithm, generates the NN candidates, and constructs computation graphs for GPU execution. We have also implemented cross-model optimization and multi-GPU optimization on the CPU-GPU system to create a more competitive baseline. The GPU power is measured using the nvidia-smi utility, and the CPU power is measured via Intel RAPL MSRs [33].

NASA. We evaluated a NASA implementation with 64 accelerator tiles. Each accelerator tile contains a 16×16 PE array and a 256 KiB SRAM buffer. The L2 buffer is 8 MiB and the incorporated HBM provides 16 GiB capacity and 512 GB/s bandwidth. The search unit contains 32 EUs and 32 KiB of SRAM dedicated to the genome buffer. OST in the mapping unit has 10k entries, and costs 128 KiB SRAM in total.

TABLE I: One-shot NAS benchmark configurations.

Benchmark	N_{block}	N_{choice}	N_{input}	Search Space
ENNAS	7	8	5	cell-based
SPOS	20	4	1	chain-structure
FairNAS	16	6	1	chain-structure

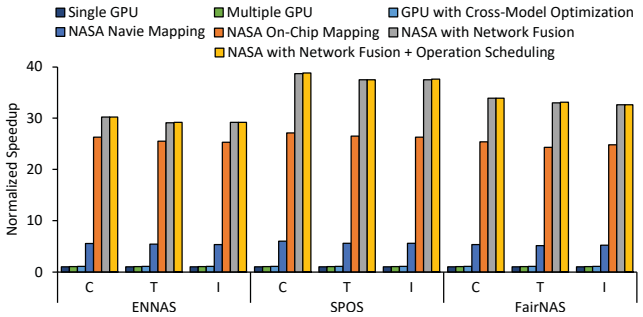


Fig. 11: Speedup of NASA compared the baseline CPU-GPU system.

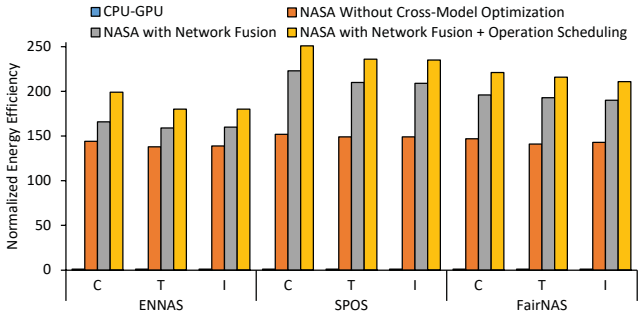


Fig. 12: Energy efficiency of NASA when compared the baseline CPU-GPU system.

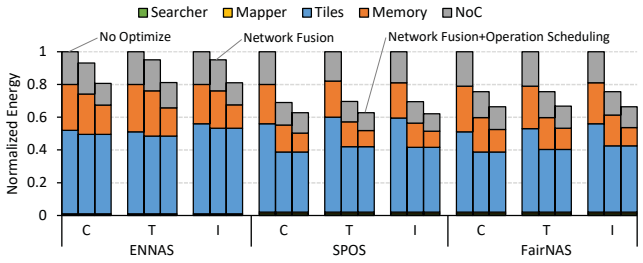


Fig. 13: Energy breakdown of NASA with different optimization.

We implemented the NASA design in RTL and synthesized it under TSMC 28nm technology with Synopsys Design Compiler. For system-level performance, we implemented a cycle-accuracy architecture simulator, cross-verified with the RTL implementation, to evaluate the chosen benchmarks on NASA. The DRAM access latency and power estimate is obtained from DRAMSim3 [37], which is ported to our simulator. According to the synthesis report, NASA occupies $38mm^2$ area, and the estimated power is $14W$ when clocked at 1 GHz.

B. Performance

Figure 11 compares the speed of all the benchmarks on the CPU-GPU system and NASA implementations. We at first evaluate the effects of cross-model optimization and multi-GPU optimization on the CPU-GPU baseline. Cross-model optimization on GPU brings a marginal performance improvement of less than 10%, which is attributed to the runtime overhead of network fusion stage as the new performance bottleneck. In addition, the multi-GPU solution brought a speedup of less than 5%. According to performance profiling, the explanation is that one-shot NAS has to frequently transfer and deploy the newly generated models from CPU to GPUs in evaluation iterations, which severely undermines the utility of the GPU resources. Also, to analyze the effects of the proposed on-chip mapping strategy in detail, We implement a naive NASA accelerator without the mapping optimization. In this situation, NASA achieves 5.46× speedup over the CPU-GPU system on average, while the PE array utility in this implementation is as low as 20%. Then We evaluate the basic NASA system that turns-off all the scheduling and fusion level optimization. In this situation, NASA achieves 24.3× to 27.1× speedup (average 25.7×) over the CPU-GPU system. Thanks to the on-chip mapping unit, the basic NASA can quickly convert new NN candidates to accelerator instructions and explore the out-of-order parallel execution of CBs. In contrast, the CPU-GPU baseline needs to construct NN models in the software framework with CPU and send it to GPU for execution, and frequently switching NN models significantly reduces GPU utilization.

Then we apply network fusion to the NASA accelerator running the same benchmarks as the baseline. The network fusion achieves an average speedup of 33.52× when compared to the CPU-GPU baseline by eliminating the redundant computation across networks. The network fusion effect varies from benchmark to benchmark, and it depends on the search space of the NAS frameworks, which impacts the cross-model computation sharing rate. For example, the acceleration effects brought by network fusion is significant for the chain-structure search space, but becomes less rewarding for the cell-based search space. The explanation is that the chance of computation sharing in the cell-based search space only exists in the first cell of the network, which means there is less redundant computation than the chain-structure search space.

Last of all, we measure the performance impact of operation scheduling. It is seen that operation scheduling brought marginal performance speed-up to NASA. The reason is that the NN operations in the design space of all benchmarks are mostly compute-intensive convolutions, the memory-level performance improvement brought by operation scheduling is often hidden by the computation overhead with the execution pipeline of prefetch-compute-store.

C. Energy Efficiency

In this experiment, We show how many images each solution could process given a fixed amount of energy, i.e., images/J, as a metric of energy efficiency. For this metric, NASA achieves an average improvement of 214.33× over the CPU-GPU system, as shown in Figure 12. The major source of the energy efficiency boost is from the performance speedup, as shown in Figure 11. Besides, it is also because NASA consumes an order of magnitude lower power than the CPU-GPU baseline.

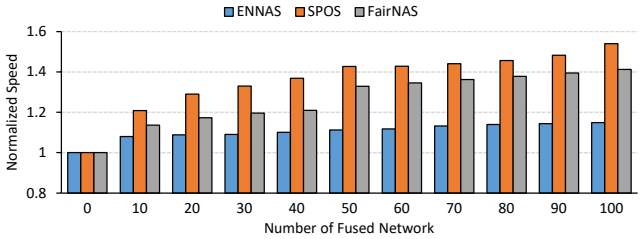


Fig. 14: Effectiveness of cross-model optimizations using different number of networks participating in network fusion.

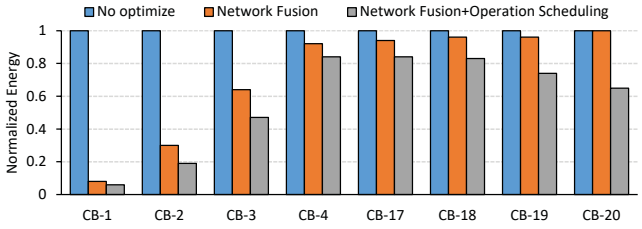
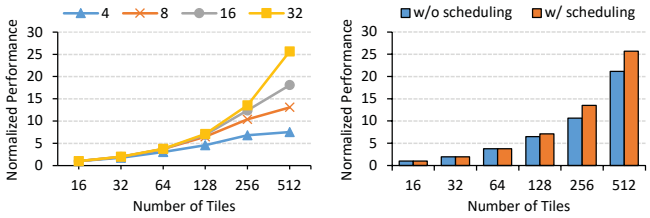


Fig. 15: Effectiveness of different cross-model optimizations on different CB of SPOS.

In order to analyze the impact of the cross-model optimization on energy consumption, we show the energy breakdown of NASA without cross-model optimization, NASA with network fusion only, and NASA, respectively in Figure 13. It can be observed that the network fusion and operation scheduling improve the average energy efficiency by 31.03% and 13.07%, respectively. Particularly, operation scheduling saves considerable memory energy because it enhances on-chip data reuse and reduces the accesses to the DRAM, while network fusion mitigates the energy spent on all the components due to the removal of the redundant operations across the NN candidates. Besides, we can find the NASA accelerator spends most of the energy on the evaluation unit, and the search unit and mapping unit consume very little energy (less than a NN accelerator tile), though they help reduce the system energy significantly.

D. Factors Affecting Cross-Network Optimization

Since the effects of cross-model computation and data sharing are closely relevant to the number of networks participating in network fusion, we analyze the influence of the network population size on NASA by changing the number of the generated networks in a search iteration. In evolutionary search, the recommended population size in each generation is around 50, so we evaluate the schemes with the number of fused networks ranging from 0 to 100. The analysis result is shown in Figure 14. It proves that higher speedup can be achieved when given a larger network population. The reason is that more network architectures sent to the fusion unit may expose more computation sharing opportunities and thus higher performance improvement. Thanks to the bypass logic and the hash table, it takes negligible time for the network fusion to locate redundant CBs from many networks. However, the shortcoming of a large-size network population is that it may cause overflow in the on-chip tables such as the unique



(a) Hardware scalability. (b) Operation Scheduling.

Fig. 16: Effectiveness of NASA using different numbers of tiles and batch sizes.

gene table and incur additional network mapping overhead. Besides, the number of fused networks is also constrained by the viable population size in the design of evolutionary algorithms. From the perspective of NASA, it encourages the use of evolutionary algorithms with a large population.

We also give a detailed performance analysis at the block-level to demonstrate the effectiveness of cross-model optimizations in different CBs of the one-shot model. In this experiment, we show how cross-model optimization reduces the energy spent on the chosen 8 CBs of SPOS at different positions of the supernet: CB-1, 2, 3, 4 at the beginning of the network, and CB-17, 18, 19, 20 at the end of the network. We adopt a population size of 50 and fuse all networks in a search iteration. Each of the shown results is an average of 30 iterations. The comparison between different schemes, including NASA without cross-model optimization, NASA with network fusion, and NASA, is shown in Figure 15. It can be observed that network fusion is more useful at the beginning of the network, and operation scheduling affects all CBs. For CB-1, since there are only four-choice CB can choose from, and inputs to the CB-1 of different networks are identical, NASA need only to process 4 CBs in practice instead of all the 50 CBs from the population of network candidates. Therefore, the computation overhead is reduced to 7.6%. For CB-2, the situation is similar to CB-1. For CB-4, which is close to the middle layers of the network, there is less computation redundancy to exploit, thereby cross-model data reuse becomes relatively more effective. For CB-20 at the end of networks, computation sharing disappears completely, and only cross-model data reuse can take effects.

E. Scalability

To evaluate the design scalability, we analyze the normalized performance of NASA under different tile sizes and batch sizes. The experimental results are shown in Figure 16(a). In general, the performance improves almost linearly with the increasing tile and batch size. However, if the batch size does not grow at a comparable rate to that of the tile numbers, the performance will not steadily improve. The reason is mainly because the computing resource in a tile will be under-utilized when the tile cannot be assigned with sufficient input instances from the partitioned batch. Besides, the fixed HBM bandwidth can also be part of the reason. When the computation time is significantly reduced due to

the growth of computing resources, the data access overhead cannot be completely hidden by the computation time slots and it will stall the pipeline. In this situation, operation scheduling, which reduces DRAM accesses through data reuse, can help alleviate this issue. Figure 16(b) shows the effects of operation scheduling when the tile size scales up from 16 to 512 with a batch size of 32. When the tile size is smaller than 128, bandwidth is not the bottleneck, and operation scheduling has a trivial impact on performance. However, when the tile size reaches 128, memory bandwidth becomes a bottleneck, and consequently the performance improvement gained by operation scheduling grows by 21.49% due to the reduction of DRAM accesses.

VII. Related Work

NAS. NAS exceeds the performance of manually designed neural architectures on many deep learning tasks [10], [12], [53], [72]. Early NAS methods [71] require mass NN training and are particularly time-consuming on large datasets. Hence, many methods have been proposed to reduce the lengthy training time [4], [64], [72]. Weight sharing strategy [51], [65] that has all the NN candidates to inherit parameters from the same supernet makes the run-time acceptable for NAS on large datasets like ImageNet [11] for the first time. Inspired by the weight sharing strategy, one-shot search [6], [9], [20]–[22], [24], [30] that separates the supernet training from network search reduces the search time to tens of GPU hours on ImageNet.

DNN Accelerator. The great success of deep neural networks has motivated a lot of DNN accelerator with 1D inner-product PE array [28], [62] or 2D spatial PE array [15], [26], [29], [34], [61]. To exploit the data reuse and computation parallelism within a single neural network, various optimized dataflows are proposed [1], [14], [15], [18], [26], [41], [44]. Recent works have also FPGA-based [31], [38], [44], [52], [68], [70] and ReRAM-based DNN [19], [55], [57], [66] accelerator. To achieve high-throughput inference of multiple neural networks, tiled architectures [18], [29], [56], [61] that consist of an array of DNN accelerators connected with NoC are proposed.

All motioned DNN accelerators rely on off-line compilation to exploit the computation parallelism and data reuse [13]. Thereby, NAS cannot be deployed on these accelerators directly. There are also several accelerators that focus on dynamic neural networks. AI-MT [2] proposed an architecture for multi-network execution through an on-chip load balancing scheduler. However, it still relies on an off-line compiler to parse each NN model and cannot handle dynamically changing models of NAS. GeneSys [54] proposed a neuro-evolutionary (NE) system to perform automated NN topology generation and training. In contrast to GeneSys that focuses on the NE algorithms and shallow NN models, NASA is oriented towards the latest NAS frameworks, which are proved effective in complicated network architecture search and generation.

VIII. Conclusion

NAS is a critical AI technique for automated neural architecture design, but it needs to evaluate a large number of dynamically generated NN candidates and it is extremely time-consuming on general purposed processors. To address this issue, we propose NASA, a specialized architecture for NAS acceleration. It has a search unit to generate NN candidates according to the specified search strategy, an mapping unit to efficiently parse, fuse and schedule the dynamically generated NNs onto an evaluation unit for high-throughput evaluation. Particularly, we observe that there are considerable fine-grained computing and data reuse opportunities among the NN candidates. Based on the observation, we further exploit the opportunities, and leverage them to remove the redundant computation and enhance the on-chip data reuse through on-chip scheduling. When compared to conventional CPU-GPU solutions, NASA achieves significant performance and energy efficiency boost.

Acknowledgement

This work was supported by the National Natural Science Foundation of China (under Grants 61876173, 61902375), and the Strategic Priority Research Program of Chinese Academy of Science (under Grant XDC05030201).

References

[1] M. Alwani, H. Chen, M. Ferdman, and P. Milder, "Fused-layer cnn accelerators," in *Proceedings of the Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2016, pp. 1–12.

[2] E. Baek, D. Kwon, and J. Kim, "A multi-neural network acceleration architecture," in *Proceedings of the ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 940–953.

[3] B. Baker, O. Gupta, N. Naik, and R. Raskar, "Designing neural network architectures using reinforcement learning," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2017.

[4] B. Baker, O. Gupta, R. Raskar, and N. Naik, "Accelerating neural architecture search using performance prediction," *arXiv preprint arXiv:1705.10823*, 2017.

[5] I. Bello, B. Zoph, V. Vasudevan, and Q. V. Le, "Neural optimizer search with reinforcement learning," in *Proceedings of the International Conference on Machine Learning (ICML)*, vol. 70. PMLR, 2017, pp. 459–468.

[6] G. Bender, P.-J. Kindermans, B. Zoph, V. Vasudevan, and Q. Le, "Understanding and simplifying one-shot architecture search," in *Proceedings of the International Conference on Machine Learning (ICML)*, vol. 80. PMLR, 2018, pp. 550–559.

[7] G. Bender, H. Liu, B. Chen, G. Chu, S. Cheng, P.-J. Kindermans, and Q. V. Le, "Can weight sharing outperform random architecture search? an investigation with tunas," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 14 323–14 332.

[8] A. Brock, T. Lim, J. Ritchie, and N. Weston, "Smash: One-shot model architecture search through hypernetworks," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2018.

[9] H. Cai, C. Gan, T. Wang, Z. Zhang, and S. Han, "Once-for-all: Train one network and specialize it for efficient deployment," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2020.

[10] H. Cai, J. Yang, W. Zhang, S. Han, and Y. Yu, "Path-level network transformation for efficient architecture search," in *Proceedings of the International Conference on Machine Learning (ICML)*, vol. 80. PMLR, 2018, pp. 678–687.

[11] H. Cai, L. Zhu, and S. Han, "Proxylessnas: Direct neural architecture search on target task and hardware," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2019.

[12] L.-C. Chen, M. Collins, Y. Zhu, G. Papandreou, B. Zoph, F. Schroff, H. Adam, and J. Shlens, "Searching for efficient multi-scale architectures for dense image prediction," in *Proceedings of the International Conference on Neural Information Processing Systems (NeurIPS)*. Curran Associates Inc., 2018, pp. 8699–8710.

[13] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy, "Tvm: An automated end-to-end optimizing compiler for deep learning," in *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 2018, pp. 578–594.

[14] T. Chen, Z. Du, N. Sun, J. Wang, C. Wu, Y. Chen, and O. Temam, "Diannao: A small-footprint high-throughput accelerator for ubiquitous machine-learning," in *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2014, pp. 269–284.

[15] Y.-H. Chen, J. Emer, and V. Sze, "Eyeriss: a spatial architecture for energy-efficient dataflow for convolutional neural networks," in *Proceedings of the ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2016, pp. 367–379.

[16] Y. Chen, G. Meng, Q. Zhang, S. Xiang, C. Huang, L. Mu, and X. Wang, "Renas: Reinforced evolutionary neural architecture search," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 4787–4796.

[17] Y. Chen, T. Yang, X. Zhang, G. Meng, X. Xiao, and J. Sun, "Detnas: Backbone search for object detection," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32. Curran Associates, Inc., 2019, pp. 6638–6648.

[18] Y. Chen, T. Luo, S. Liu, S. Zhang, L. He, J. Wang, L. Li, T. Chen, Z. Xu, N. Sun, and O. Temam, "Dadiannao: A machine-learning supercomputer," in *Proceedings of the Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2014, pp. 609–622.

[19] P. Chi, S. Li, C. Xu, T. Zhang, J. Zhao, Y. Liu, Y. Wang, and Y. Xie, "Prime: a novel processing-in-memory architecture for neural network computation in reram-based main memory," in *Proceedings of the ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2016, pp. 27–39.

[20] X. Chu, X. Wang, B. Zhang, S. Lu, X. Wei, and J. Yan, "Darts-: robustly stepping out of performance collapse without indicators," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[21] X. Chu, B. Zhang, R. Xu, and J. Li, "Fairnas: Rethinking evaluation fairness of weight sharing neural architecture search," *arXiv preprint arXiv:1907.01845*, 2019.

[22] X. Chu, T. Zhou, B. Zhang, and J. Li, "Fair darts: Eliminating unfair advantages in differentiable architecture search," in *Proceedings of the European Conference on Computer Vision (ECCV)*. Springer, 2020, pp. 465–480.

[23] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2009, pp. 248–255.

[24] X. Dong and Y. Yang, "One-shot neural architecture search via self-evaluated template network," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2019, pp. 3681–3690.

[25] X. Dong and Y. Yang, "Searching for a robust neural architecture in four gpu hours," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 1761–1770.

[26] Z. Du, R. Fasthuber, T. Chen, P. Ienne, L. Li, T. Luo, X. Feng, Y. Chen, and O. Temam, "Shidiannao: Shifting vision processing closer to the sensor," in *Proceedings of the ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*. ACM, 2015, pp. 92–104.

[27] T. Elsken, J. H. Metzen, and F. Hutter, "Efficient multi-objective neural architecture search via lamarckian evolution," in *Procedings of the International Conference on Learning Representations (ICLR)*, 2019.

[28] J. Fowers, K. Ovtcharov, M. Papamichael, T. Massengill, M. Liu, D. Lo, S. Alkalay, M. Haselman, L. Adams, M. Ghandi, S. Heil, P. Patel, A. Sapek, G. Weisz, L. Woods, S. Lanka, , S. K. Reinhardt, A. M. Caulfield, E. S. Chung, and D. Burger, "A configurable cloud-scale dnn processor for real-time ai," in *Proceedings of the ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2018, pp. 1–14.

[29] M. Gao, X. Yang, J. Pu, M. Horowitz, and C. Kozyrakis, "Tangram: Optimized coarse-grained dataflow for scalable nn accelerators," in *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2019, pp. 807–820.

[30] Z. Guo, X. Zhang, H. Mu, W. Heng, Z. Liu, Y. Wei, and J. Sun, "Single path one-shot neural architecture search with uniform sampling," in *Proceedings of the European Conference on Computer Vision (ECCV)*. Springer, 2020, pp. 544–560.

[31] S. Han, J. Kang, H. Mao, Y. Hu, X. Li, Y. Li, D. Xie, H. Luo, S. Yao, Y. Wang, H. Yang, and W. J. Dally, "Ese: Efficient speech recognition engine with sparse lstm on fpga," in *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*. ACM, 2017, pp. 75–84.

[32] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.

[33] Intel Corporation, *Intel® 64 and IA-32 Architectures Software Developer's Manual Volume 3B: System Programming Guide, Part 2*, 2011.

[34] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers, R. Boyle, P. luc Cantin, C. Chao, C. Clark, J. Coriell, M. Daley, M. Dau, J. Dean, B. Gelb, T. V. Ghaemmaghami, R. Gottipati, W. Gulland, R. Hagmann, C. R. Ho, D. Hogberg, J. Hu, R. Hundt, D. Hurt, J. Ibarz, A. Jaffey, A. Jaworski, A. Kaplan, H. Khaitan, D. Killebrew, A. Koch, N. Kumar, S. Lacy, J. Laudon, J. Law, D. Le, C. Leary, Z. Liu, K. Lucke, A. Lundin, G. MacKean, A. Maggiore, M. Mahony, K. Miller, R. Nagarajan, R. Narayanaswami, R. Ni, K. Nix, T. Norrie, M. Omernick, N. Penukonda, A. Phelps, J. Ross, M. Ross, A. Salek, E. Samadiani, C. Severn, G. Sizikov, M. Snelham, J. Souter, D. Steinberg, A. Swing, M. Tan, G. Thorson, B. Tian, H. Toma, E. Tuttle, V. Vasudevan, R. Walter, W. Wang, E. Wilcox, and D. H. Yoon, "In-datacenter performance analysis of a tensor processing unit," in *Proceedings of the ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*. ACM, 2017, pp. 1–12.

[35] A. Krizhevsky, "Learning multiple layers of features from tiny images," 2009.

[36] L. Li and A. Talwalkar, "Random search and reproducibility for neural architecture search," in *Proceedings of the Uncertainty in Artificial Intelligence Conference (UAI)*, vol. 115. PMLR, 2019, pp. 367–377.

[37] S. Li, Z. Yang, D. Reddy, A. Srivastava, and B. Jacob, "Dramsim3: a cycle-accurate, thermal-capable dram simulator," *IEEE Computer Architecture Letters*, vol. 19, no. 2, pp. 106–109, 2020.

[38] Z. Li, C. Ding, S. Wang, W. Wen, Y. Zhuo, C. Liu, Q. Qiu, W. Xu, X. Lin, X. Qian, and Y. Wang, "E-rnn: Design optimization for efficient recurrent neural networks in fpgas," in *Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2019, pp. 69–80.

[39] C. Liu, L.-C. Chen, F. Schroff, H. Adam, W. Hua, A. L. Yuille, and L. Fei-Fei, "Auto-deeplab: Hierarchical neural architecture search for semantic image segmentation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 82–92.

[40] H. Liu, K. Simonyan, and Y. Yang, "Darts: Differentiable architecture search," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2019.

[41] W. Lu, G. Yan, J. Li, S. Gong, Y. Han, and X. Li, "Flexflow: A flexible dataflow accelerator architecture for convolutional neural networks," in *Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2017, pp. 553–564.

[42] Z. Lu, K. Deb, E. Goodman, W. Banzhaf, and V. N. Boddeti, "Nsganetv2: Evolutionary multi-objective surrogate-assisted neural architecture search," in *Proceedings of the European Conference on Computer Vision (ECCV)*. Springer, 2020, pp. 35–51.

[43] Z. Lu, I. Whalen, V. Boddeti, Y. Dhebar, K. Deb, E. Goodman, and W. Banzhaf, "Nsga-net: neural architecture search using multi-objective genetic algorithm," in *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO)*. ACM, 2019, pp. 419–427.

[44] Y. Ma, Y. Cao, S. Vrudhula, and J.-s. Seo, "Optimizing loop operation and dataflow in fpga acceleration of deep convolutional neural networks," in *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*. ACM, 2017, pp. 45–54.

[45] B. L. Miller and D. E. Goldberg, "Genetic algorithms, tournament selection, and the effects of noise," *Complex systems*, vol. 9, no. 3, pp. 193–212, 1995.

[46] NVIDIA Corporation. (2017) Nvidia deep learning accelerator. [Online]. Available: http://nvdla.org/

[47] NVIDIA Corporation. (2018) Jetson agx xavier — nvidia developer. [Online]. Available: https://developer.nvidia.com/embedded/jetson-agx-xavier

[48] NVIDIA Corporation. (2018) Nvidia deep learning sdk. [Online]. Available: https://docs.nvidia.com/deeplearning/sdk/index.html

[49] NVIDIA Corporation. (2019) Cuda toolkit documentation v10.1.243. [Online]. Available: https://docs.nvidia.com/cuda/archive/10.1/

[50] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32. Curran Associates, Inc., 2019, pp. 8026–8037.

[51] H. Pham, M. Guan, B. Zoph, Q. Le, and J. Dean, "Efficient neural architecture search via parameters sharing," in *Proceedings of the International Conference on Machine Learning (ICML)*, vol. 80. PMLR, 2018, pp. 4095–4104.

[52] J. Qiu, J. Wang, S. Yao, K. Guo, B. Li, E. Zhou, J. Yu, T. Tang, N. Xu, S. Song, Y. Wang, and H. Yang, "Going deeper with embedded fpga platform for convolutional neural network," in *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*. ACM, 2016, pp. 26–35.

[53] E. Real, A. Aggarwal, Y. Huang, and Q. V. Le, "Regularized evolution for image classifier architecture search," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, vol. 33, 2019, pp. 4780–4789.

[54] A. Samajdar, P. Mannan, K. Garg, and T. Krishna, "Genesys: Enabling continuous learning through neural network evolution in hardware," in *Proceedings of the Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2018, pp. 855–866.

[55] A. Shafiee, A. Nag, N. Muralimanohar, R. Balasubramonian, J. P. Strachan, M. Hu, R. S. Williams, and V. Srikumar, "Isaac: a convolutional neural network accelerator with in-situ analog arithmetic in crossbars," in *Proceedings of the ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2016, pp. 14–26.

[56] Y. S. Shao, J. Clemons, R. Venkatesan, B. Zimmer, M. Fojtik, N. Jiang, B. Keller, A. Klinefelter, N. Pinckney, P. Raina, S. G. Tell, Y. Zhang, W. J. Dally, J. Emer, C. T. Gray, B. Khailany, and S. W. Keckler, "Simba: Scaling deep-learning inference with multi-chip-module-based architecture," in *Proceedings of the Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. ACM, 2019, pp. 14–27.

[57] L. Song, X. Qian, H. Li, and Y. Chen, "Pipelayer: A pipelined reram-based accelerator for deep learning," in *Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2017, pp. 541–552.

[58] M. Suganuma, S. Shirakawa, and T. Nagao, "A genetic programming approach to designing convolutional neural network architectures," in *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO)*. ACM, 2017, pp. 497–504.

[59] M. Tan, B. Chen, R. Pang, V. Vasudevan, M. Sandler, A. Howard, and Q. V. Le, "Mnasnet: Platform-aware neural architecture search for mobile," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 2820–2828.

[60] TinyImageNet. (2020) Tiny imagenet visual recognition challenge. [Online]. Available: https://tiny-imagenet.herokuapp.com/

[61] S. Venkataramani, A. Ranjan, S. Banerjee, D. Das, S. Avancha, A. Jagannathan, A. Durg, D. Nagaraj, B. Kaul, P. Dubey, and A. Raghunathan, "Scaledeep: A scalable compute architecture for learning and evaluating deep networks," in *Proceedings of the ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*. ACM, 2017, pp. 13–26.

[62] R. Venkatesan, Y. S. Shao, M. Wang, J. Clemons, S. Dai, M. Fojtik, B. Keller, A. Klinefelter, N. Pinckney, P. Raina, Y. Zhang, B. Zimmer, W. J. Dally, J. S. Emer, S. W. Keckler, and B. Khailany, "Magnet: A modular accelerator generator for neural networks," in *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 2019, pp. 1–8.

[63] A. Wan, X. Dai, P. Zhang, Z. He, Y. Tian, S. Xie, B. Wu, M. Yu, T. Xu, K. Chen, P. Vajda, and J. E. Gonzalez, "Fbnetv2: Differentiable neural architecture search for spatial and channel dimensions," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 12 965–12 974.

[64] T. Wei, C. Wang, Y. Rui, and C. W. Chen, "Network morphism," in *Proceedings of the International Conference on Machine Learning (ICML)*, vol. 48. PMLR, 2016, pp. 564–572.

[65] L. Xie, X. Chen, K. Bi, L. Wei, Y. Xu, Z. Chen, L. Wang, A. Xiao, J. Chang, X. Zhang *et al.*, "Weight-sharing neural architecture search: A battle to shrink the optimization gap," *arXiv preprint arXiv:2008.01475*, 2020.

[66] T.-H. Yang, H.-Y. Cheng, C.-L. Yang, I.-C. Tseng, H.-W. Hu, H.-S. Chang, and H.-P. Li, "Sparse reram engine: Joint exploration of activation and weight sparsity in compressed neural networks," in *Proceedings of the ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*. ACM, 2019, pp. 236–249.

[67] S. You, T. Huang, M. Yang, F. Wang, C. Qian, and C. Zhang, "Greedynas: Towards fast one-shot nas with greedy supernet," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 1996–2005.

[68] C. Zhang, P. Li, G. Sun, Y. Guan, B. Xiao, and J. Cong, "Optimizing fpga-based accelerator design for deep convolutional neural networks," in *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*. ACM, 2015, pp. 161–170.

[69] M. Zhang, H. Li, S. Pan, T. Liu, and S. Su, "One-shot neural architecture search via novelty driven sampling," in *Proceedings of the International Joint Conference on Artiicial Intelligence (IJCAI)*. International Joint Conferences on Artificial Intelligence Organization, 2020, pp. 3188–3194.

[70] X. Zhang, J. Wang, C. Zhu, Y. Lin, J. Xiong, W.-m. Hwu, and D. Chen, "Dnnbuilder: an automated tool for building high-performance dnn hardware accelerators for fpgas," in *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 2018, pp. 1–8.

[71] B. Zoph and Q. V. Le, "Neural architecture search with reinforcement learning," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2017.

[72] B. Zoph, V. Vasudevan, J. Shlens, and Q. V. Le, "Learning transferable architectures for scalable image recognition," in *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 8697–8710.